

习题五

一、选择题

1. 设有一个 10 阶的对称矩阵 A, 采用压缩存储方式, 以行序为主存储, a_{11} 为第一元素, 其存储地址为 1, 每个元素占一个地址空间, 则 a_{85} 的地址为 ()。
A. 13 B. 33 C. 18 D. 40
2. 有一个二维数组 A[1:6, 0:7] 每个数组元素用相邻的 6 个字节存储, 存储器按字节编址, 那么这个数组的体积是 (①) 个字节。假设存储数组元素 A[1, 0] 的第一个字节的地址是 0, 则存储数组 A 的最后一个元素的第一个字节的地址是 (②)。若按行存储, 则 A[2, 4] 的第一个字节的地址是 (③)。若按列存储, 则 A[5, 7] 的第一个字节的地址是 (④)。就一般情况而言, 当 (⑤) 时, 按行存储的 A[i, j] 地址与按列存储的 A[j, i] 地址相等。供选择的①-④: A. 12 B. 66 C. 72 D. 96 E. 114 F. 120
G. 156 H. 234 I. 276 J. 282 K. 283 L. 288
⑤: A. 行与列的上界相同 B. 行与列的下界相同
C. 行与列的上、下界都相同 D. 行的元素个数与列的元素个数相同
3. 将一个 A[1..100, 1..100] 的三对角矩阵, 按行优先存入一维数组 B[1..298] 中, A 中元素 a_{6665} (即该元素下标 $i=66, j=65$), 在 B 数组中的位置 K 为 ()。供选择的答案:
A. 198 B. 195 C. 197
4. 二维数组 A 的每个元素是由 6 个字符组成的串, 其行下标 $i=0,1,\dots,8$, 列下标 $j=1,2,\dots,10$ 。若 A 按行先存储, 元素 A[8,5] 的起始地址与当 A 按列先存储时的元素 () 的起始地址相同。设每个字符占一个字节。
A. A[8,5] B. A[3,10] C. A[5,8] D. A[0,9]
5. 若对 n 阶对称矩阵 A 以行序为主序方式将其下三角形的元素(包括主对角线上所有元素)依次存放于一维数组 B [1..(n(n+1)/2)] 中, 则在 B 中确定 a_{ij} ($i < j$) 的位置 k 的关系为 ()。
A. $i*(i-1)/2+j$ B. $j*(j-1)/2+i$ C. $i*(i+1)/2+j$ D. $j*(j+1)/2+i$
6. 有一个 100*90 的稀疏矩阵, 非 0 元素有 10 个, 设每个整型数占 2 字节, 则用三元组表示该矩阵时, 所需的字节数是 ()。
A. 60 B. 66 C. 18000 D. 33
7. 数组 A[0..4, -1..-3, 5..7] 中含有元素的个数 ()。
A. 55 B. 45 C. 36 D. 16
8. 用数组 r 存储静态链表, 结点的 next 域指向后继, 工作指针 j 指向链中结点, 使 j 沿链移动的操作为 ()。
A. $j=r[j].next$ B. $j=j+1$ C. $j=j \rightarrow next$ D. $j=r[j] \rightarrow next$
9. 对稀疏矩阵进行压缩存储目的是 ()。
A. 便于进行矩阵运算 B. 便于输入和输出
C. 节省存储空间 D. 降低运算的时间复杂度
10. 已知广义表 L = ((x,y,z), a, (u, t, w)), 从 L 表中取出原子项 t 的运算是 ()。
A. head (tail (tail (L))) B. tail (head (head (tail (L))))
C. head (tail (head (tail (L)))) D. head (tail(head (tail (tail (L)))))
11. 广义表 A=(a,b,(c,d),(e,(f,g))), 则下面式子的值为 ()。
Head(Tail(Head(Tail(Tail(A)))))
A. (g) B. (d) C. c D. d
12. 设广义表 L = ((a,b,c)), 则 L 的长度和深度分别为 ()。
A. 1 和 1 B. 1 和 3 C. 1 和 2 D. 2 和 3
13. 下面说法不正确的是 ()。
A. 广义表的表头总是一个广义表 B. 广义表的表尾总是一个广义表
C. 广义表难以用顺序存储结构 D. 广义表可以是一个多层次的结构

二、判断题

1. 数组不适合作为任何二叉树的存储结构。 ()

2. 从逻辑结构上看, n 维数组的每个元素均属于 n 个向量。()
3. 稀疏矩阵压缩存储后, 必会失去随机存取功能。()
4. 数组是同类型值的集合。()
5. 数组可看成线性结构的一种推广, 因此与线性表一样, 可以对它进行插入, 删除等操作。()
6. 一个稀疏矩阵 $A_{m \times n}$ 采用三元组形式表示, 若把三元组中有关行下标与列下标的值互换, 并把 m 和 n 的值互换, 则就完成了 $A_{m \times n}$ 的转置运算。()
7. 二维以上的数组其实是一种特殊的广义表。()
8. 广义表的取表尾运算, 其结果通常是个表, 但有时也可是个单元元素值。()
9. 若一个广义表的表头为空表, 则此广义表亦为空表。()
10. 广义表中的元素或者是一个不可分割的原子, 或者是一个非空的广义表。()
11. 所谓取广义表的表尾就是返回广义表中最后一个元素。()
12. 广义表的同级元素 (直属于同一个表中的各元素) 具有线性关系。()
13. 对长度为无穷大的广义表, 由于存储空间限制, 不能在计算机中实现。()
14. 一个广义表可以为其它广义表所共享。()

三、填空题

1. 数组的存储结构采用_____存储方式。
2. 设二维数组 $A[-20..30, -30..20]$, 每个元素占有 4 个存储单元, 存储起始地址为 200.如按行优先顺序存储,则元素 $A[25,18]$ 的存储地址为(1); 如按列优先顺序存储,则元素 $A[-18,-25]$ 的存储地址为(2)。
3. 设数组 $a[1..50, 1..80]$ 的基地址为 2000, 每个元素占 2 个存储单元, 若以行序为主序顺序存储, 则元素 $a[45,68]$ 的存储地址为(1);若以列序为主序顺序存储, 则元素 $a[45,68]$ 的存储地址为(2)。
4. 三维数组 $a[4][5][6]$ (下标从 0 开始计, a 有 $4 \times 5 \times 6$ 个元素), 每个元素的长度是 2, 则 $a[2][3][4]$ 的地址是____。(设 $a[0][0][0]$ 的地址是 1000,数据以行为主方式存储)
5. 用一维数组 B 与列优先存放带状矩阵 A 中的非零元素 $A[i,j]$ ($1 \leq i \leq n, i-2 \leq j \leq i+2$), B 中的第 8 个元素是 A 中的第(1)行, 第(2)列的元素。
6. 设数组 $A[0..8, 1..10]$, 数组中任一元素 $A[i,j]$ 均占内存 48 个二进制位, 从首地址 2000 开始连续存放在主内存里, 主内存字长为 16 位, 那么
 - (1) 存放该数组至少需要的单元数是_____;
 - (2) 存放数组的第 8 列的所有元素至少需要的单元数是_____;
 - (3) 数组按列存储时, 元素 $A[5,8]$ 的起始地址是_____。
7. 设 n 行 n 列的下三角矩阵 A 已压缩到一维数组 $B[1..n \times (n+1) / 2]$ 中, 若按行为主序存储, 则 $A[i,j]$ 对应的 B 中存储位置为_____。
8. 已知三对角矩阵 $A[1..9, 1..9]$ 的每个元素占 2 个单元, 现将其三条对角线上的元素逐行存储在起始地址为 1000 的连续的内存单元中, 则元素 $A[7,8]$ 的地址为_____。
9. 当广义表中的每个元素都是原子时, 广义表便成了_____。
10. 广义表简称表, 是由零个或多个原子或子表组成的有限序列, 原子与表的差别仅在于(1)。为了区分原子和表, 一般用(2)表示表, 用(3)表示原子。一个表的长度是指(4), 而表的深度是指(5)。
11. 设广义表 $L = ((), ())$, 则 $head(L)$ 是(1); $tail(L)$ 是(2); L 的长度是(3); 深度是(4)。
12. 已知广义表 $A = (9, 7, (8, 10, (99)), 12)$, 试用求表头和表尾的操作 $Head()$ 和 $Tail()$ 将原子元素 99 从 A 中取出来。
13. 已知广义表 $A = (((a, b), (c), (d, e)))$, $head(tail(tail(head(A))))$ 的结果是_____。

四、应用题

1. 设 $m \times n$ 阶稀疏矩阵 A 有 t 个非零元素, 其三元组表表示为 $LTMA[1..(t+1), 1..3]$, 试问: 非零元素的个数 t 达到什么程度时用 $LTMA$ 表示 A 才有意义?
2. 对一个有 t 个非零元素的 A_{mn} 矩阵, 用 $B[0..t][1..3]$ 的数组来表示, 其中第 0 行的三个元素分别为 m, n, t , 从第一行开始到最后一行, 每行表示一个非零元素; 第一列为矩阵元素的行号, 第二列为其列号, 第三列为其值。对这样的表示法, 如果需要经常进行该操作---确定任意一个元素 $A[i][j]$ 在 B 中的位置并修改其值, 应如何设计算法可以使时间得到改善?

3. 设有三对角矩阵 $(a_{ij})_{m \times n}$,将其三条对角线上的元素逐行的存于数组 B(1:3n-2)中, 使得 $B[k]=a_{ij}$, 求:

(1) 用 i,j 表示 k 的下标变换公式;

(2) 若 $n=10^3$,每个元素占用 L 个单元, 则用 $B[K]$ 方式比常规存储节省多少单元。

4. 画出下列广义表的两种存储结构图($()$,A,(B,(C,D)), $()$,E,F)。

五 算法设计题

1. 请编写完整的程序。如果矩阵 A 中存在这样的元素 $A[i,j]$ 满足条件: $A[i,j]$ 是第 i 行中值最小的元素,且又是第 j 列中值最大的元素, 则称之为该矩阵的一个马鞍点。请编程计算出 $m \times n$ 的矩阵 A 的所有马鞍点。

2. 给定一个整数数组 $b[0..N-1]$, b 中连续的相等元素构成的子序列称为平台。试设计算法, 求出 b 中最长平台的长度。

3. 给定 $n \times m$ 矩阵 $A[a..b,c..d]$,并设 $A[i,j] \leq A[i,j+1](a \leq i \leq b, c \leq j \leq d-1)$ 和 $A[i,j] \leq A[i+1,j](a \leq i \leq b-1, c \leq j \leq d)$.设计一算法判定 X 的值是否在 A 中,要求时间复杂度为 $O(m+n)$ 。

4. 设二维数组 $a[1..m, 1..n]$ 含有 $m \times n$ 个整数。

(1) 写出算法(c 函数): 判断 a 中所有元素是否互不相同?输出相关信息(yes/no);

(2) 试分析算法的时间复杂度。

5. 试编写建立广义表存储结构的算法, 要求在输入广义表的同时实现判断、建立。设广义表按如下形式输入 $(a_1,a_2,a_3,\dots,a_n)$ $n \geq 0$, 其中 a_i 或为单字母表示的原子或为广义表, $n=0$ 时为只含空格字符的空表。(注:算法可用类 pascal 或类 c 书写)

6. 设 $A[1..100]$ 是一个记录构成的数组, $B[1..100]$ 是一个整数数组, 其值介于 1 至 100 之间, 现要求按 $B[1..100]$ 的内容调整 A 中记录的次序, 比如当 $B[1]=11$ 时, 则要求将 $A[1]$ 的内容调整到 $A[11]$ 中去。规定可使用的附加空间为 $O(1)$ 。

参考答案

一、选择题

1.B	2.1L	2.2J	2.3C	2.4I	2.5C	3.B	4. B	5.B
6.B	7.B	8.A	9.C	10.D	11.D	12.C	13.A	

二、判断题

1.×	2.√	3. √	4. ×	5. ×	6. ×	7. √
8.×	9. ×	10. ×	11. ×	12.√	13.√	14.√

三、填空题

1.顺序存储结构

2. (1) 9572 (2) 1228

3. (1) 9174 (2) 8788

4. 1164 公式: $LOC(a_{ijk})=LOC(a_{000})+[v_2 \times v_3 \times (i-c_1)+v_3 \times (j-c_2)+(k-c_3)] \times l$ (l 为每个元素所占单元数)

5.第 1 行第 3 列

6. (1)270 (2)27 (3)2204

7. $i(i-1)/2+j (1 \leq i,j \leq n)$

8. 1038 三对角矩阵按行存储: $k=2(i-1)+j (1 \leq i,j \leq n)$

9. 线性表

10. (1) 原子(单元素)是结构上不可再分的, 可以是一个数或一个结构; 而表带结构, 本质就是广义表, 因作为广义表的元素故称为子表。

(2) 大写字母 (3) 小写字母 (4) 表中元素的个数 (5) 表展开后所含括号的层数

11. (1) $()$ (2) $(())$ (3) 2 (4) 2

12. head (head (tail (tail (head (tail (tail (A)))))))

13. (d,e)

四 应用题

1.稀疏矩阵 A 有 t 个非零元素, 加上行数 mu 、列数 nu 和非零元素个数 tu (也算一个三元组), 共占用三元组表 LTMA 的 $3(t+1)$ 个存储单元, 用二维数组存储时占用 $m \times n$ 个单元, 只有当 $3(t+1) < m \times n$ 时, 用 LTMA 表示 A 才有意

义。解不等式得 $t < m \cdot n / 3 - 1$ 。

2. 题中矩阵非零元素用三元组表存储, 查找某非零元素时, 按常规要从第一个元素开始查找, 属于顺序查找, 时间复杂度为 $O(n)$ 。若使查找时间得到改善, 可以建立索引, 将各行行号及各行第一个非零元素在数组 B 中的位置 (下标) 偶对放入一向量 C 中。若查找非零元素, 可先在数组 C 中用折半查找到该非零元素的行号, 并取出该行第一个非零元素在 B 中的位置, 再到 B 中顺序 (或折半) 查找该元素, 这时时间复杂度为 $O(\log n)$ 。

3. (1) $k = 3(i-1)$ (主对角线左下角, 即 $i=j+1$)

$k = 3(i-1)+1$ (主对角线上, 即 $i=j$)

$k = 3(i-1)+2$ (主对角线上, 即 $i=j-1$)

由以上三式, 得 $k = 2(i-1)+j$ ($1 \leq i, j \leq n; 1 \leq k \leq 3n-2$)

(2) $10^3 \cdot 10^3 - (3 \cdot 10^3 - 2)$

4. 广义表的第一种存储结构的理论基础是, 非空广义表可唯一分解成表头和表尾两部分, 而由表头和表尾可唯一构成一个广义表。这种存储结构中, 原子和表采用不同的结点结构 (“异构”, 即结点域个数不同)。

原子结点两个域: 标志域 $tag=0$ 表示原子结点, 域 DATA 表示原子的值; 子表结点三个域: $tag=1$ 表示子表, hp 和 tp 分别是指向表头和表尾的指针。在画存储结构时, 对非空广义表不断进行表头和表尾的分解, 表头可以是原子, 也可以是子表, 而表尾一定是表 (包括空表)。上面是本题的第一种存储结构图。

广义表的第二种存储结构的理论基础是, 非空广义表最高层元素间具有逻辑关系: 第一个元素无前驱有后继, 最后一个元素无后继有前驱, 其余元素有唯一前驱和唯一后继。有人将这种结构看作扩充线性结构。这种存储结构中, 原子和表均采用三个域的结点结构 (“同构”)。结点中都有一个指针域指向后继结点。原子结点中还包括标志域 $tag=0$ 和原子值域 DATA; 子表结点还包括标志域 $tag=1$ 和指向子表的指针 hp 。在画存储结构时, 从左往右一个元素一个元素的画, 直至最后一个元素。下面是本题的第二种存储结构图。

五 算法设计题

1. [题目分析] 寻找马鞍点最直接的方法, 是在一行中找出一个最小值元素, 然后检查该元素是否是元素所在列的最大元素, 如是, 则输出一个马鞍点, 时间复杂度是 $O(m \cdot (m+n))$ 。本算法使用两个辅助数组 max 和 min , 存放每列中最大值元素的行号和每行中最小值元素的列号, 时间复杂度为 $O(m \cdot n + m)$, 但比较次数比前种算法会增加, 也多使用向量空间。

```
int m=10, n=10;
void Saddle(int A[m][n])
//A 是 m*n 的矩阵, 本算法求矩阵 A 中的马鞍点.
{int max[n]={0}, //max 数组存放各列最大值元素的行号, 初始化为行号 0;
  min[m]={0}, //min 数组存放各行最小值元素的列号, 初始化为列号 0;
  i, j;
  for(i=0; i<m; i++) //选各行最小值元素和各列最大值元素.
    for(j=0; j<n; j++)
      {if(A[max[j]][j]<A[i][j]) max[j]=i; //修改第 j 列最大元素的行号
        if(A[i][min[i]]>A[i][j]) min[i]=j; //修改第 i 行最小元素的列号.
      }
  for (i=0; i<m; i++)
    {j=min[i]; //第 i 行最小元素的列号
      if(i==max[j])printf("A[%d][%d]是马鞍点, 元素值是%d", i, j, A[i][j]); //是马鞍点
    }
}
```

[算法讨论] 以上算法假定每行 (列) 最多只有一个可能的马鞍点, 若有多个马鞍点, 因为一行 (或一列) 中可能的马鞍点数值是相同的, 则可用二维数组 $min2$, 第一维是行向量, 是各行行号, 第二维是列向量, 存放一行中最大值的列号。对最大值也同样处理, 使用另一二维数组 $max2$, 第一维是列向量, 是各列列号, 第二维存该列最大值元素的行号。最后用类似上面方法, 找出每行 (i) 最小值元素的每个列号 (j), 再到 $max2$ 数组中找该列是否有最大值元素的行号 (i), 若有, 则是马鞍点。

2. [题目分析] 我们用 l 代表最长平台的长度, 用 k 指示最长平台在数组 b 中的起始位置 (下标)。用 j 记住局部平台的起始位置, 用 i 指示扫描 b 数组的下标, i 从 0 开始, 依次和后续元素比较, 若局部平台长度 $(i-j)$ 大于 l 时, 则

修改最长平台的长度 k ($l=i-j$) 和其在 b 中的起始位置 ($k=j$), 直到 b 数组结束, l 即为所求。

```
void Platform (int b[], int N)
//求具有 N 个元素的整型数组 b 中最长平台的长度。
{ l=1; k=0; j=0; i=0;
  while(i<n-1)
  { while(i<n-1 && b[i]==b[i+1]) i++;
    if(i-j+1>l) { l=i-j+1; k=j; } //局部最长平台
    i++; j=i; } //新平台起点
  printf("最长平台长度%d, 在 b 数组中起始下标为%d", l, k);
}
```

3.[题目分析]矩阵中元素按行和按列都已排序, 要求查找时间复杂度为 $O(m+n)$, 因此不能采用常规的二层循环的查找。可以先从右上角 ($i=a, j=d$) 元素与 x 比较, 只有三种情况: 一是 $A[i,j]>x$, 这情况下向 j 小的方向继续查找; 二是 $A[i,j]<x$, 下步应向 i 大的方向查找; 三是 $A[i,j]=x$, 查找成功。否则, 若下标已超出范围, 则查找失败。

```
void search(datatype A[][ ], int a,b,c,d, datatype x)
//n*m 矩阵 A,行下标从 a 到 b,列下标从 c 到 d,本算法查找 x 是否在矩阵 A 中.
{ i=a; j=d; flag=0; //flag 是成功查到 x 的标志
  while(i<=b && j>=c)
  { if(A[i][j]==x) { flag=1; break; }
    else if (A[i][j]>x) j--; else i++;
    if(flag) printf("A[%d][%d]=%d", i,j,x); //假定 x 为整型.
    else printf("矩阵 A 中无%d 元素", x);
  }
  printf("算法 search 结束。");
}
```

[算法讨论]算法中查找 x 的路线从右上角开始, 向下 (当 $x>A[i,j]$) 或向左 (当 $x<A[i,j]$)。向下最多是 m , 向左最多是 n 。最佳情况是在右上角比较一次成功, 最差是在左下角 ($A[b,c]$), 比较 $m+n$ 次, 故算法最差时间复杂度是 $O(m+n)$ 。

4. [题目分析]判断二维数组中元素是否互不相同, 只有逐个比较,找到一对相等的元素, 就可结论为不是互不相同。如何达到每个元素同其它元素比较一次且只一次? 在当前行, 每个元素要同本行后面的元素比较一次 (下面第一个循环控制变量 p 的 for 循环), 然后同第 $i+1$ 行及以后各行元素比较一次, 这就是循环控制变量 k 和 p 的二层 for 循环。

```
int JudgEqual(int a[m][n],int m,n)
//判断二维数组中所有元素是否互不相同, 如是, 返回 1; 否则, 返回 0。
{ for(i=0; i<m; i++)
  { for(j=0; j<n-1; j++)
    { for(p=j+1; p<n; p++) //和同行其它元素比较
      if(a[i][j]==a[i][p]) { printf("no"); return(0); }
    //只要有一个相同的, 就结论不是互不相同
    for(k=i+1; k<m; k++) //和第 i+1 行及以后元素比较
      for(p=0; p<n; p++)
        if(a[i][j]==a[k][p]) { printf("no"); return(0); }
    }
  }
  printf("yes"); return(1); //元素互不相同
}
```

(2) 二维数组中的每一个元素同其它元素都比较一次, 数组中共 $m*n$ 个元素, 第 1 个元素同其它 $m*n-1$ 个元素比较, 第 2 个元素同其它 $m*n-2$ 个元素比较, ..., 第 $m*n-1$ 个元素同最后一个元素($m*n$)比较一次,所以在元素互不相等时总的比较次数为 $(m*n-1)+(m*n-2)+\dots+2+1=(m*n)(m*n-1)/2$ 。在有相同元素时,可能第一次比较就相同,也可能最后一次比较时相同,设在 $(m*n-1)$ 个位置上均可能相同,这时的平均比较次数约为 $(m*n)(m*n-1)/4$, 总的时间复杂度是 $O(n^4)$

5. [题目分析] 广义表的元素有原子和表。在读入广义表“表达式”时, 遇到左括号‘(’就递归的构造子表, 否则若是原

子，就建立原子结点；若读入逗号‘,’，就递归构造后续子表；若 n=0，则构造含空格字符的空表，直到碰到输入结束符号（‘#’）。设广义表的形式定义如下：

```
typedef struct node
{int tag;                //tag=0 为原子，tag=1 为子表
 struct node *link;      //指向后继结点的指针
 union {struct node *slink; //指向子表的指针
        char data;        //原子
        }element;
}Glist;
Glist *creat ()
//建立广义表的存储结构
{char ch; Glist *gh;
 scanf("%c",&ch);
 if(ch=="") gh=null;
 else {gh=(Glist*)malloc(sizeof(Glist));
       if(ch=='('){gh->tag=1; //子表
                  gh->element.slink=creat(); } //递归构造子表
       else {gh->tag=0;gh->element.data=ch;} //原子结点
       }
 scanf("%c",&ch);
 if(gh!=null) if(ch==',') gh->link=creat(); //递归构造后续广义表
               else gh->link=null;
 return(gh);
}
```

}算法结束

6. [题目分析]题目要求按 B 数组内容调整 A 数组中记录的次序，可以从 i=1 开始，检查是否 B[i]=i。如是，则 A[i]恰为正确位置，不需再调；否则，B[i]=k≠i，则将 A[i]和 A[k]对调，B[i]和 B[k]对调，直到 B[i]=i 为止。

```
void CountSort (rectype A[],int B[])
//A 是 100 个记录的数组，B 是整型数组，本算法利用数组 B 对 A 进行计数排序
{int i,j,n=100;
 i=1;
 while(i<n)
 {if(B[i]!=i) //若 B[i]=i 则 A[i]正好在自己的位置上，则不需要调整
  { j=i;
   while (B[j]!=i)
    { k=B[j]; B[j]=B[k]; B[k]=k; // B[j]和 B[k]交换
      r0=A[j];A[j]=A[k]; A[k]=r0; } //r0 是数组 A 的元素类型,A[j]和 A[k]交换
   i++;} //完成了一个小循环，第 i 个已经安排好
 } //算法结束
```